

SIWY/WIMU - DOKUMENTACJA KOŃCOWA

Jakub Proboszcz

Paweł Kochański

Mikołaj Piórczyński

jakub.proboszcz.stud@pw.edu.pl

pawel.kochanski3.stud@pw.edu.pl

mikolaj.piorczynski.stud@pw.edu.pl

1. TEMAT PROJEKTU

Atrybucja danych to zadanie polegające na określeniu, które przykłady ze zbioru uczącego miały największy wpływ na predykcję modelu dla danego przykładu testowego. Celem projektu było zbadanie problemu atrybucji w dziedzinie audio, a szczególności muzyki, wychodząc od metody TRAK, która jak dotąd nie była jeszcze zastosowana do tej modalności.

2. ANALIZA LITERATURY

Metoda TRAK została zaproponowana w artykule [1] jako narzędzie atrybucji danych treningowych dla predykcji modeli uczenia maszynowego. Autorzy zaprezentowali jej zastosowanie w kilku zadaniach, w tym w klasyfikacji obrazów, klasyfikacji tekstu, obliczaniu embeddingów z wykorzystaniem modelu CLIP oraz w zadaniu generacji zamaskowanych fragmentów tekstu za pomocą modeli językowych. Implementacja metody jest publicznie dostępna poprzez bibliotekę `trak` dostępną w repozytorium GitHub¹, jednak nie zawiera kompletnej obsługi wyjść modeli w zadaniach generacji tekstu.

Zastosowania metody TRAK w dziedzinie audio są jak dotąd bardzo ograniczone. Znaleźliśmy jedynie jedną pracę wykorzystującą TRAK w tym kontekście [2]. Autorzy analizują wpływ danych treningowych na modele autoregresywne generujące muzykę w formacie MIDI. W eksperymentach wykorzystano modele Anticipatory Music Transformer uczony na zbiorze MAESTRO [3] oraz Music Transformer [4] uczony na zbiorze TheoryTab.

Znacznie więcej badań dotyczy zastosowań metody TRAK w dziedzinie przetwarzania obrazów, w szczególności w kontekście modeli dyfuzyjnych. Przykładem jest praca D-TRAK [5], w której zaproponowano szereg modyfikacji oryginalnej metody TRAK dostosowujących ją do specyfiki modeli dyfuzyjnych. Eksperymenty przeprowadzono na zbiorach danych CIFAR oraz CelebA z wykorzystaniem własnych modeli DDPM, a także na zbiorze ArtBench przy użyciu dotrenowanego modelu Stable Diffusion.

Kolejną pracą badającą atrybucję danych w modelach dyfuzyjnych jest Journey-TRAK [6]. Autorzy stosują w niej bazową wersję metody TRAK, koncentrując się na analizie wpływu danych treningowych na różnych etapach procesu odszumiania. Badania obejmują własne modele DDPM uczone na zbiorze CIFAR-10 oraz modele LDM trenowane na zbiorze MS COCO.

¹ <https://github.com/MadryLab/trak>

Metoda TRAK znalazła również zastosowanie w analizie dużych modeli językowych. W pracy [7] zaproponowano metodę TrackStar. Autorzy uczą własne modele na zbiorze danych C4, a ewaluacja jest wykonywana na zbiorze T-REx.

Alternatywne podejście do atrybucji danych w modelach generujących audio zaproponowano w pracy [8], w której problem ten analizowany jest z perspektywy unlearningu. Autorzy badają warianty polegające zarówno na usuwaniu próbek treningowych, jak i próbek wygenerowanych przez model. Eksperymenty przeprowadzono z wykorzystaniem własnego modelu opartego na architekturze Stable Audio, uczonego na własnym zbiorze danych.

3. DOPRECYZOWANIE TEMATU

Jako konkretne zadanie do rozwiązania w ramach naszego projektu zostało ustalone uruchomienie metody TRAK na dwóch modelach generujących audio: modelu autoregresywnym MusicGen Small [9] oraz modelu dyfuzyjnym Stable Audio Open Small [10]. Jako zbiór danych wykorzystywany do dotrenowywania modeli został wybrany zbiór danych MusicCaps [11].

4. METODOLOGIA

4.1 Zbiór danych

Zbiór danych MusicCaps jest dostępny jako plik CSV zawierający ID filmiku na YouTube z którego pochodzi audio wraz z momentem początkowym, końcowym i opisem. Pobraliśmy zbiór danych samodzielnie; po usunięciu próbek błędnych, niedostępnych już na YouTube, nietrwających dokładnie 10 sekund, zawierających więcej niż 2 kanały uzyskaliśmy 4484 próbki, spośród których wydzieliliśmy 915 jako zbiór testowy.

4.2 Stable Audio Open Small

Na modelu Stable Audio Open Small, badany był jeden wariant funkcji wyjścia modelu: strata dyfuzji, oparta na $\mathcal{L}_{simple}$ z D-TRAK [5]:

$$f(y) = \sum_{t \in \mathcal{T}} \|\epsilon - x(\sqrt{\alpha}y + \sqrt{1 - \alpha}\epsilon, t)\|_2^2 \quad (1)$$

gdzie:

- $\mathcal{T}$ to zbiór 6 losowo wybranych (z rozkładu $\sigma(\mathcal{N}(0, 1))$ przyciętego do przedziału $[0.075, 1]$) wartości,

- ϵ to szum z rozkładu $\mathcal{N}(0, 1)$,
- $x(\sqrt{\bar{\alpha}}y + \sqrt{1 - \bar{\alpha}}\epsilon, t)$ to predykcja szumu zwrócona przez model.

Funkcja gradientu straty do funkcji wartości to stała 1.

4.3 MusicGen Small

Na modelu MusicGen Small, pomimo wielu prób, nie udało się osiągnąć satysfakcjonujących (zauważalnie lepszych od losowego zgadywania) wyników powyżej opisanego testu. Sprawdzone były warianty funkcji wyjścia modelu: straty modelu MusicGen (cross-entropy), klasyfikacji wieloklasowej pierwszego tokenu oraz sumy klasyfikacji wieloklasowej tokenów.

W pierwszym wariancie, strata modelu MusicGen jest liczona między logitami zwróconymi przez model a tokenami z ocenianej próbki. Wynik jest sumowany po wszystkich tokenach oraz po 4 codebookach. W tym wariancie gradient straty po funkcji wyjścia modelu to stała 1. Wzór przedstawia się następująco (funkcja "loss"):

$$f(y) = \sum_{t=1}^T \sum_{k=1}^K \log \left(\frac{\exp(x_{tk}[y_{tk}])}{\sum_{i=1}^C \exp(x_{tk}[i])} \right) \quad (2)$$

gdzie:

- $x_{tk}[i]$ - logit dla t -tego tokenu, k -tego codebooka, wartości i ,
- y_{tk} - oczekiwana wartość dla t -tego tokenu, k -tego codebooka,
- $C = 2048$ - liczba możliwych wartości w każdym z codebooków,
- $K = 4$ - liczba codebooków,
- $T = 500$ - liczba tokenów w 10-sekundowym audio.

Drugi wariant jest analogiczny dla przypadku klasyfikacji wieloklasowej i został zaimplementowany możliwie podobnie do funkcji wyjścia „text_classification” z biblioteki trak. W tym przypadku rozważany problem to klasyfikacja wieloklasowa pierwszego tokenu, a funkcja wyjścia modelu jest taka, jak dla klasyfikacji wieloklasowej (funkcja "single"):

$$f(y) = \log \left(\frac{p_{00}[y_{00}]}{1 - p_{00}[y_{00}]} \right) \quad (3)$$

gdzie $p_{00}[i]$ to prawdopodobieństwo poprawnej wartości dla pierwszego tokenu i codebooka uzyskane na podstawie logitów za pomocą funkcji softmax z temperaturą równą 1. W tym wariancie gradient straty do funkcji wyjścia to $1 - p_{00}$.

Trzecim sprawdzanym wariantem była funkcja wyjścia jako suma wartości obliczonych jak wyżej dla wszystkich tokenów i codebooków, o następującym wzorze (funkcja "summed"):

$$f(y) = \sum_{t=1}^T \sum_{k=1}^K \log \left(\frac{p_{tk}[y_{tk}]}{1 - p_{tk}[y_{tk}]} \right) \quad (4)$$

W tym wariancie występuje problem: strata (suma entropii krzyżowych po codebookach i tokenach) nie jest

funkcją funkcji wyjścia modelu, więc nie da się policzyć gradientu straty do funkcji wyjścia. Testowane były dwa warianty: suma wartości dla klasyfikacji wieloklasowej, czyli $\sum_{t=1}^T \sum_{k=1}^K (1 - p_{tk})$, oraz stała 1.

Wszystkie powyższe warianty były sprawdzane dla dwóch sposobów obliczania logitów: podstawowego obliczania logitów warunkowanych opisem ocenianego audio (jak podczas treningu), albo obliczania logitów korzystając z Classifier-Free Generation (jak podczas generacji) jako:

$$\text{logit}_{\text{uncond}} + \text{cfg_coef}(\text{logit}_{\text{cond}} - \text{logit}_{\text{uncond}}) \quad (5)$$

gdzie cfg_coef dla MusicGen Small wynosi 3.

4.4 Test implementacji

W związku z problemami z obiektywną ewaluacją atrybucji danych, przygotowaliśmy test sprawdzający sensowność implementacji. Polega on na wytrenowaniu zo-
verfitowanego modelu na małym podzbiorze danych treningowych (u nas 64 próbki). Na takim pojedynczym, modelu następnie obliczany jest TRAK. Jako wygenerowane próbki, które mają być wyjaśnione, podawane są próbki wygenerowane na podstawie opisów próbek treningowych, lub same próbki treningowe. Oczekiwany wynik atrybucji danych jest wtedy znany – jest to macierz jednostkowa, czyli każda próbka powinna zostać przypisana do siebie samej. Mierzmy liczbę, ile razy w zbiorze 64 przykładów dla danej próbki ta sama próbka została oceniona najwyżej (top-1 accuracy) i ile razy znalazła się wśród 5 najwyższych ocen (top-5 accuracy). Wyniki tych miar dla losowego zgadywania byłyby podobne odpowiednio do $\frac{1}{64} \approx 0.02$ i $\frac{5}{64} \approx 0.08$.

Model	Wariant	CFG	top-1	top-5
SAO	—	—	0.33	0.52
MusicGen	loss	yes	0.02	0.11
MusicGen	loss	no	0.03	0.11
MusicGen	single	yes	0.03	0.08
MusicGen	single	no	0.00	0.06
MusicGen	summed	yes	0.00	0.02
MusicGen	summed	no	0.02	0.09

Tabela 1. Wyniki testu implementacji

Dla Stable Audio Open Small udało się osiągnąć wynik testu znacznie lepszy od losowego zgadywania, co potwierdza poprawność implementacji. Natomiast dla MusicGen Small, żaden z badanych wariantów nie uzyskał wyników znacząco lepszych niż losowe.

4.5 Wyniki na pełnym zbiorze MusicCaps

Uruchomiliśmy TRAK na MusicGen Small oraz Stable Audio Open Small na pełnym zbiorze MusicCaps. Wytrenowaliśmy po 100 checkpointów dla każdego modelu, pochodzących z 10 niezależnych treningów. Z każdego treningu checkpointy były zapisywane po epokach od 6 do 15.

Wyniki są ogólnie perceptualnie niezadowolające – najwyżej ocenione próbki nie są podobne do generacji. Może to wynikać ze zbyt słabego douczenia modeli na zbiorze MusicCaps – wybraliśmy takie parametry uczenia, aby train loss spadał, ale valid loss nie wzrastał, w związku z czym generacje pozostały sensowne, ale mogły być niewystarczająco zmienione przez finetuning. Jako że MusicCaps nie ma żadnego konkretnego „stylu” muzyki, ciężko stwierdzić skuteczność finetuningu.

5. IMPLEMENTACJA

Implementacja naszego projektu jest zawarta w publicznym repozytorium². W jej ramach wykorzystaliśmy również forki repozytoriów bibliotek `trak`³ oraz `stable-audio-tools`⁴.

Instrukcja instalacji, dotrenowania modeli i uruchomienia metody TRAK jest zawarta w README.md w repozytorium. Eksperymenty są realizowane przez:

- skrypty w języku Python 3.11 wykonujące właściwe obliczenia,
- skrypty shellowe wykonujące finetuning modeli, ustalające ścieżki i parametry do skryptów w Pythonie,
- polecenia w justfile wywołujące odpowiednie skrypty,
- notebook w Jupyter prezentujący najwyżej ocenione próbki treningowe dla kilku generacji.

Jakość biblioteki `trak` (nieaktualizowanej od 2 lat) pozostawia wiele do życzenia; jej niespodziewane zachowania wymagające zmian w samej bibliotece lub niestandardowego ustawienia parametrów to między innymi:

- domyślnie wykorzystywany jest `FunctionalGradientComputer`, czego niestety przez niekompatybilność mechanizmu *codebook interleaving pattern* w MusicGen z funkcją `torch.vmap` nie dało się zastosować; alternatywny `IterativeGradientComputer` zwraca inny format danych (zwykły tensor zamiast `dict[str, Tensor]`),
- pomimo domyślnego ustawienia precyzji na `float16`, `CudaProjector` wewnętrznie zwraca wynik jako `float32`, a wyniki są rzutowane na `float16`,
- przy domyślnej konfiguracji parametr `proj_max_batch_size` ustawiony jest na 32 i klasa `ChunkedCudaProjector` alokuje pamięć na 32 elementy, nawet gdy `batch_size` to 2,
- pomimo zwracania wyników jako `float32`, `CudaProjector` wewnętrznie rzutuje wejście na `float16`, co skutkowało przycinaniem się nielicznych bardzo dużych gradientów do maksymalnej wartości `float16`.

6. REFERENCES

- [1] S. M. Park, K. Georgiev, A. Ilyas, G. Leclerc, and A. Madry, “Trak: Attributing model behavior at scale,” in *International Conference on Machine Learning (ICML)*, 2023.
- [2] J. Deng, S. Zhang, and J. Ma, “Computational copyright: Towards a royalty model for music generative ai,” *arXiv preprint arXiv:2312.06646*, 2023.
- [3] C. Hawthorne, A. Stasyuk, A. Roberts, I. Simon, C.-Z. A. Huang, S. Dieleman, E. Elsen, J. Engel, and D. Eck, “Enabling factorized piano music modeling and generation with the maestro dataset,” *arXiv preprint arXiv:1810.12247*, 2018.
- [4] C.-Z. A. Huang, A. Vaswani, J. Uszkoreit, N. Shazeer, I. Simon, C. Hawthorne, A. M. Dai, M. D. Hoffman, M. Dinculescu, and D. Eck, “Music transformer,” *arXiv preprint arXiv:1809.04281*, 2018.
- [5] X. Zheng, T. Pang, C. Du, J. Jiang, and M. Lin, “Intriguing properties of data attribution on diffusion models,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] K. Georgiev, J. Vendrow, H. Salman, S. M. Park, and A. Madry, “The journey, not the destination: How data guides diffusion models,” *arXiv preprint arXiv:2312.06205*, 2023.
- [7] T. A. Chang, D. Rajagopal, T. Bolukbasi, L. Dixon, and I. Tenney, “Scalable influence and fact tracing for large language model pretraining,” 2024.
- [8] W. Choi, J. Koo, K. W. Cheuk, J. Serrà, M. A. Martínez-Ramírez, Y. Ikemiya, N. Murata, Y. Takida, W.-H. Liao, and Y. Mitsufuji, “Large-scale training data attribution for music generative models via unlearning,” *arXiv preprint arXiv:2506.18312*, 2025.
- [9] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, “Simple and controllable music generation,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 47 704–47 720, 2023.
- [10] Z. Novack, Z. Evans, Z. Zukowski, J. Taylor, C. Carr, J. Parker, A. Al-Sinan, G. M. Iodice, J. McAuley, T. Berg-Kirkpatrick *et al.*, “Fast text-to-audio generation with adversarial post-training,” *arXiv preprint arXiv:2505.08175*, 2025.
- [11] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi *et al.*, “Musiclm: Generating music from text,” *arXiv preprint arXiv:2301.11325*, 2023.

²<https://github.com/Pavilon-Mk-Git/25Z-WIMU-music-data-attribution>

³<https://github.com/JakPro15/trak/tree/main>

⁴<https://github.com/Pavilon-Mk-Git/stable-audio-tools/tree/main>